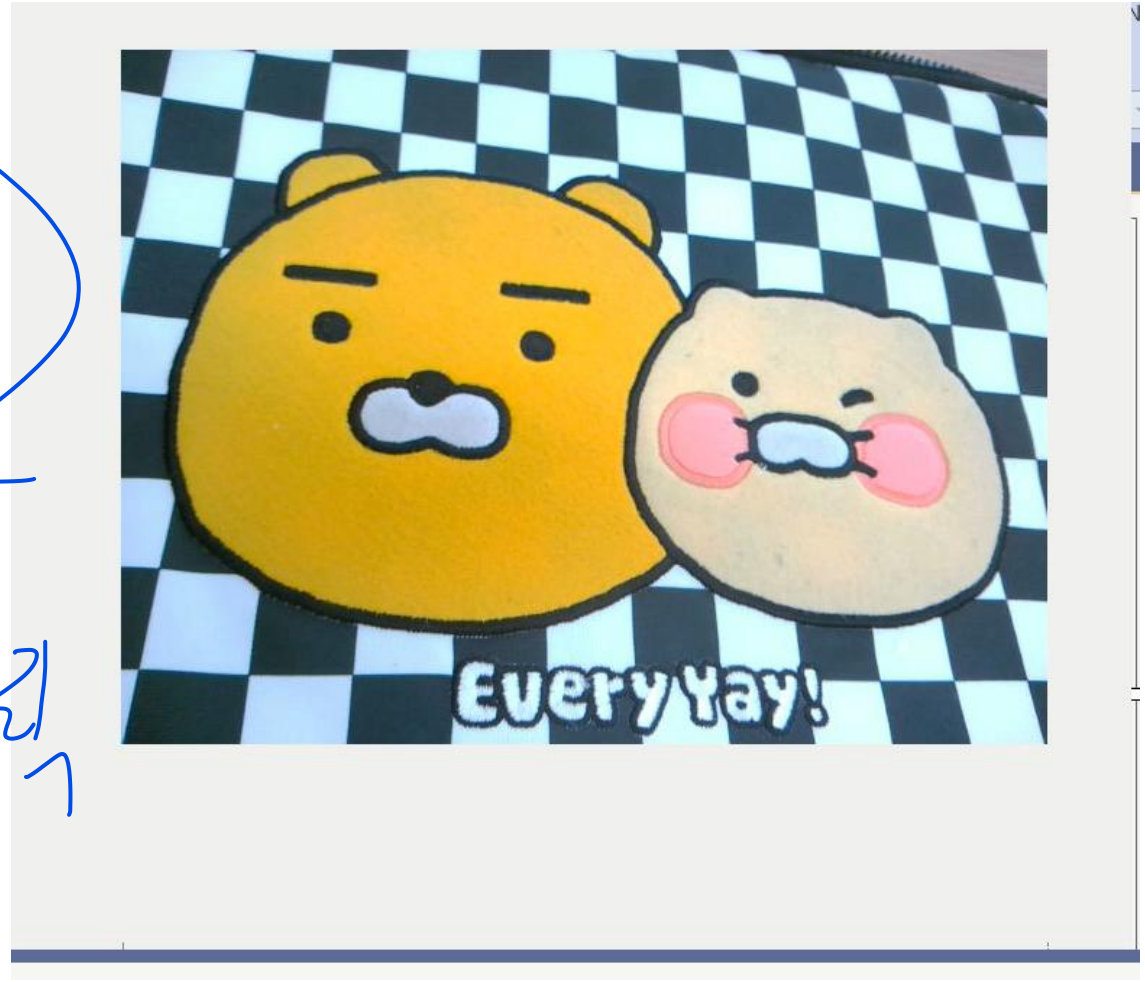
윈폼에서 OpenCV사용하기

2. 고급컨트롤

- 픽쳐박스를 통한 실시간 카메라 영상 출력
- 픽쳐박스 1개 추가
- timer 속성 및 이벤트
  - Enabled : True
  - Interval : 33(약 30fps)
  - timer1_Tick 이벤트 추가

- 폼 속성 및 이벤트
  - Size : 800, 700
  - FormBorderStyle : None(뒷장)
  - Form_Load : Form1_Load 이벤트 추가
  - FormClosing : Form1_FormCosing이벤트 추가



폼 기반의 프로그램의 이벤트

- 새프로젝트 만들기

Form1.cs [디자인]* | Form1.cs*

기본 디스플레이의 크기 조정이 125%로 설정되어 있습니다. WinForms Designer DPI-Unaware 모드로 여는 것이 좋습니다.
자세한 정보

WinForms_Control_Ex2 프로젝트가 더 이상 지원되지 않는 .NET 6.0을(를) 대상으로 하며, 이후 릴리스에서는 Windows Forms 디자이너에서 로드되지 않습니다. 지원되는 .NET 버전으로 업데이트하는 것이 좋습니다. 자세한 정보 다시 표시하지 않음

Form1

.Net 8.0

속성

Form1 System.Windows.Forms.Fo

BackgroundIm	Tile
CancelButton	(none)
CausesValidati	True
ContextMenuS	(none)
ControlBox	True
Cursor	Default
DoubleBuffere	False
Enabled	True
Font	맑은 고딕, 9pt
FormBorderSty	Sizable
Icon	(Icon)
ImeMode	NoControl
IsMdiContaine	False
KeyPreview	False

기본 디스플레이의 크기 조정이 125%로 설정되어 있습니다. WinForms Designer DPI-Unaware 모드로 여는 것이 좋습니다.
자세한 정보

WinForms_Control_Ex2 프로젝트가 더 이상 지원되지 않는 .NET 6.0을(를) 대상으로 하며, 이후 릴리스에서는 Windows Forms 디자이너에서 로드되지 않습니다. 지원되는 .NET 버전으로 업데이트하는 것이 좋습니다. 자세한 정보 다시 표시하지 않음

폼 제목 안보이게 설정

속성

Form1 System.Windows.Forms.Fo

BackgroundIm	Tile
CancelButton	(none)
CausesValidation	True
ContextMenuS	(none)
ControlBox	True
Cursor	Default
DoubleBuffere	False
Enabled	True
ForeColor	ControlText
FormBorderSty	None
Icon	(Icon)
ImeMode	NoControl
IsMdiContaine	False
KeyPreview	False
Language	(Default)
Localizable	False
Location	0, 0

FormBorderStyle
폼의 테두리 및 제목 표시줄의 모양과 동작을 나타냅니다.

도구 상자

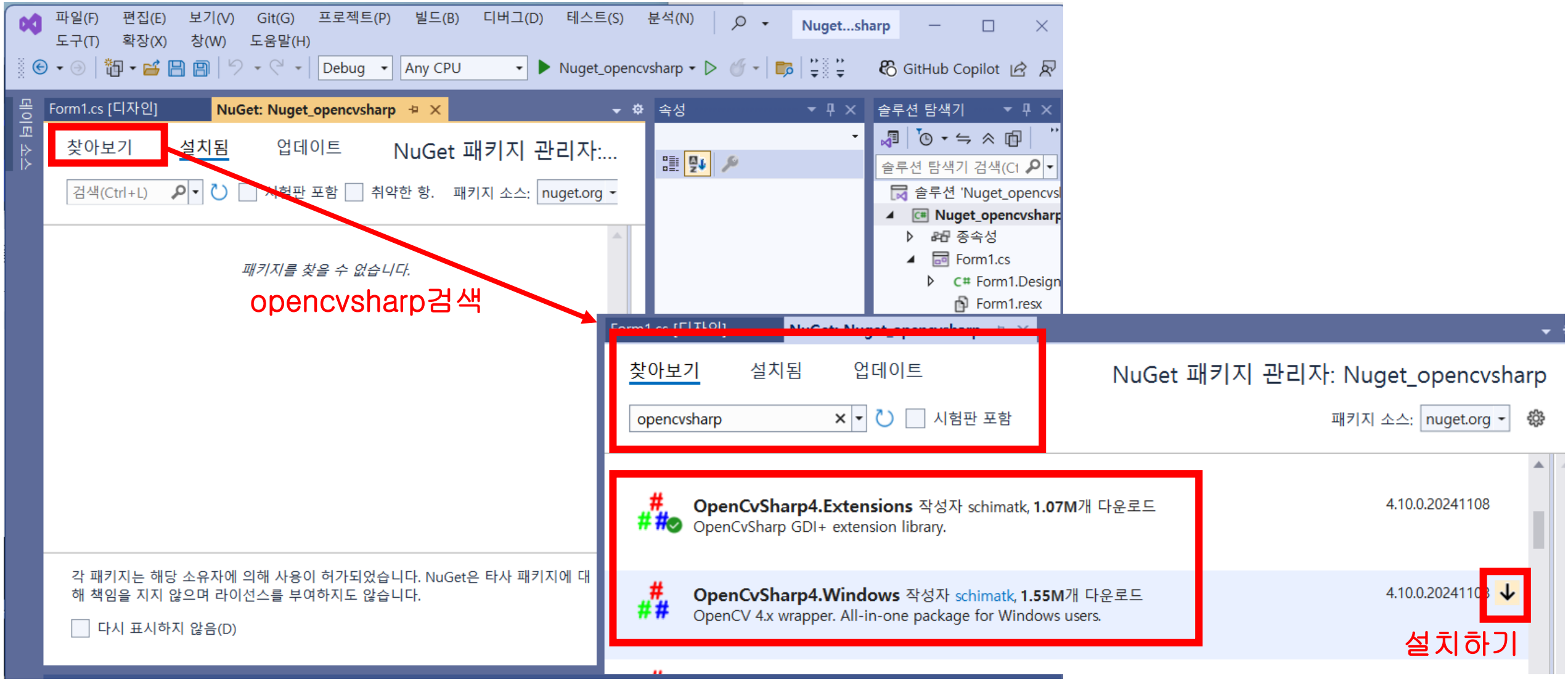
검색 도구 상자

모든 Windows Forms
- 포인터
- BackgroundWo...
- BindingSource
- Button
- CheckBox
- CheckedListBox
- ColorDialog
- ContextMenuS...
- ComboBox
- DataGridView
- DateTimePicker
- DomainUpDown
- ErrorProvider
- FileSystemWatc...
- FlowLayoutPanel
- FolderBrowser...
- FontDialog
- GroupBox
- HelpProvider
- HScrollBar

솔루션 탐색기 | 도구 상자

OpenCV를 사용(Nuget 패키지 관리)한 영상 출력

- 프로젝트 클릭 -> 우클릭 -> Nuget 패키지 관리



2. 고급컨트롤-컴파일시 카메라가 켜지는 시간이 소요됨

```
// OpenCV를 .NET에서 사용하기 위한 확장 모듈과 관련 네임스페이스 포함
using OpenCvSharp.Extensions;  //Nuget 패키지 추가
using OpenCvSharp; //Nuget 패키지 추가
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;
using System.Threading.Tasks;

namespace Nuget_opencvsharp
{
    public partial class Form1 : Form
    {
     // OpenCV의 비디오 캡처 객체 선언 (카메라 또는 동영상 파일을 통해 영상 가져옴)
        OpenCvSharp.VideoCapture video;
        // 영상 프레임을 저장할 Mat 객체 (OpenCV의 이미지 객체)
       Mat mImage = new Mat();
        public Form1()
        {
            InitializeComponent();  //디자이너에 의해 생성된 컴포넌트들을 초기화
        }
```

Mat 객체 : OpenCV의 이미지 데이터 컨테이너, ToBitmap()을 통해 WinForms에서 쓸 수 있도록 변환

2. 고급컨트롤

```
// 폼이 로드될 때 호출되는 이벤트 핸들러
private void Form1_Load(object sender, EventArgs e)
{
    try
    {
// 기본 카메라(0번 장치)를 열기
        video = new VideoCapture(0, VideoCaptureAPIs.DSHOW));
// 프레임 해상도 설정
        video.FrameWidth = 640;
        video.FrameHeight = 480;
    }
    catch
    {
// 카메라 열기에 실패한 경우 타이머 정지 (영상 캡처 중단)
MessageBox.Show("error");
timer1.Enabled = false;
    }
}
```

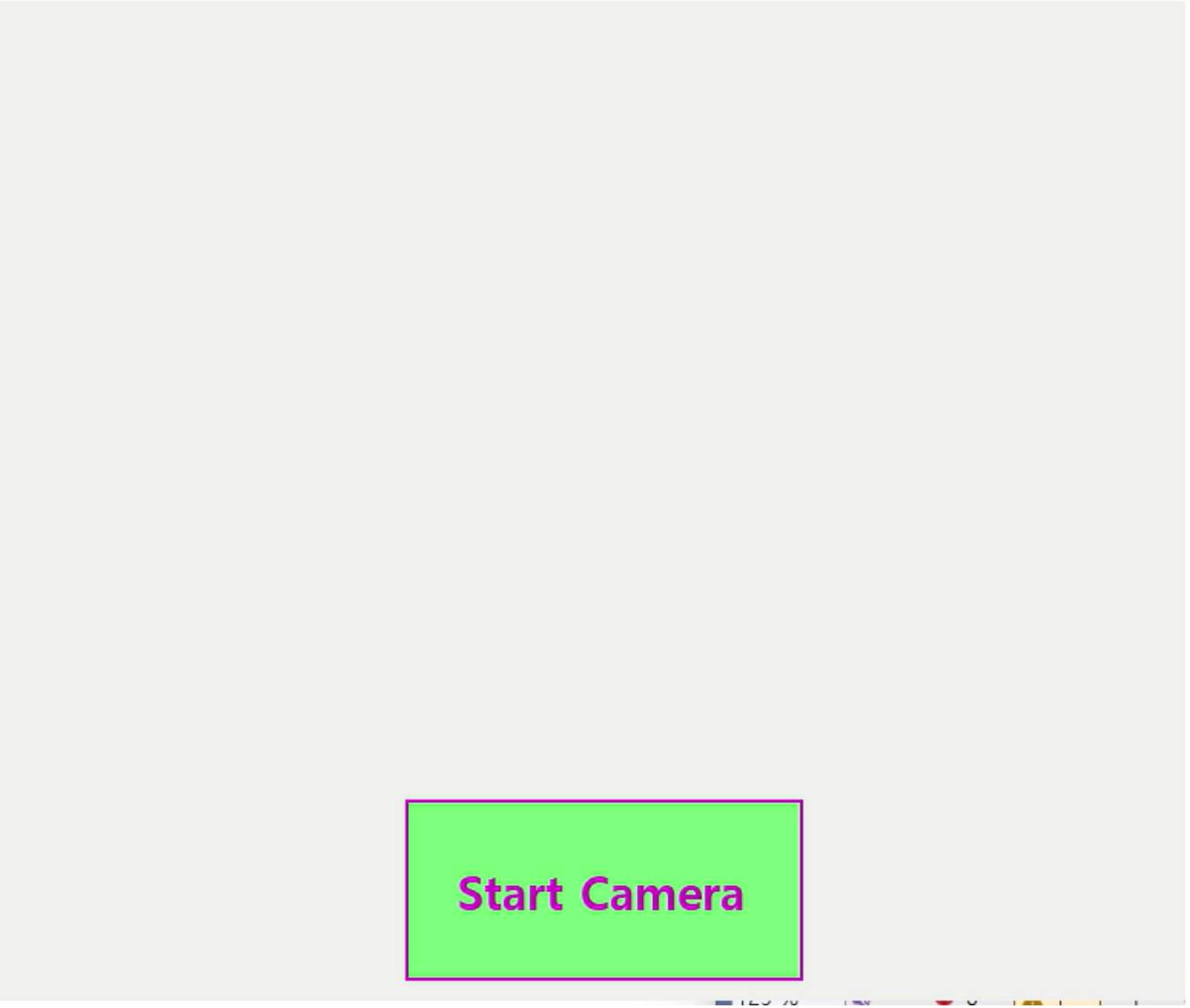
2. 고급컨트롤

```
// 타이머 이벤트 핸들러: 주기적으로 호출되어 카메라에서 프레임을 읽고 PictureBox에 출력
private void timer1_Tick(object sender, EventArgs e)
    {
        // 현재 프레임을 mImage에 저장
        video.Read(mImage);
        // Mat 객체(mImage)를 Bitmap으로 변환한 후 PictureBox에 표시
        pictureBox1.Image = OpenCvSharp.Extensions.BitmapConverter.ToBitmap(mImage);
        }

// 폼이 종료될 때 호출되는 이벤트 핸들러
private void Form1_FormClosing(object sender, FormClosingEventArgs e)
        {
        // 사용한 Mat 객체의 자원 해제 (메모리 누수 방지)
            mImage.Dispose();
            video?.Dispose();
        }
    }
}
```

버튼 클릭을 통한 카메라 on/off 전환

- 카메라 ON/OFF 전환 기능 추가
- OpenCvSharp
- 버튼 추가
  - Text : 공백 아무것도 쓰지 말라고 하지만
  - Font : 18pt,style=Bold
  - ForeColor : 사용자지정색 선택
  - FlatStyle : Flat
  - BackColor : 사용자지정색 선택
- 동기식(Timer)
  - timer_Tick 이벤트는 UI 스레드에서 질행
  - 프레임 읽기+이미지 갱신이 UI에서 수행



버튼 클릭을 통한 카메라 on/off 전환1/5

```
// OpenCV를 .NET 환경에서 사용할 수 있도록 해주는 확장 메서드 포함
using OpenCvSharp.Extensions;
using OpenCvSharp;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;
using System.Threading.Tasks;

namespace Nuget_opencvsharp
{
    public partial class Form1 : Form
    {
        // 카메라를 제어하는 VideoCapture 객체
        OpenCvSharp.VideoCapture video;
        // 영상을 담기 위한 OpenCV의 Mat 객체 (이미지 버퍼)
        Mat mImage = new Mat();
        // 카메라 실행 여부를 판단하기 위한 상태 플래그
        private bool isCameraRunning = false; // 카메라 상태 저장 변수
```

버튼 클릭을 통한 카메라 on/off 전환2/5

```
public Form1()
{
    InitializeComponent();
}

private void Form1_Load(object sender, EventArgs e)
{
 // 초기 상태: 카메라 비활성화
    // 카메라는 아직 시작하지 않으므로 텍스트로 설정
    btn_Toggle.Text = "Start Camera";
}
```

버튼 클릭을 통한 카메라 on/off 전환3/5

```
// 타이머가 주기적으로 호출되는 이벤트 핸들러
// 카메라에서 프레임을 읽어 PictureBox에 출력
private void timer1_Tick(object sender, EventArgs e)
{
        // video 객체가 null이 아니고, 카메라가 켜져 있다면
    if (video != null && video.IsOpened())
    {
        // 현재 프레임을 mImage에 읽어옴
        video.Read(mImage);
        // 읽은 이미지가 비어 있지 않으면 화면에 출력
        if (!mImage.Empty())
        {
            pictureBox1.Image = BitmapConverter.ToBitmap(mImage);
        }
    }
}
```

버튼 클릭을 통한 카메라 on/off 전환4/5

```
// 폼이 닫히기 직전에 호출되는 이벤트 핸들러
// 사용된 자원을 안전하게 정리
private void Form1_FormClosing(object sender, FormClosingEventArgs e)
{
    timer1.Stop();   // 타이머 중지 (프레임 읽기 멈춤)
    video?.Release();  // 카메라 장치 해제
    mImage.Dispose();   // Mat 메모리 해제
}
// 버튼 클릭 시 실행되는 이벤트 핸들러: 카메라 ON/OFF 토글 기능
private void btn_Toggle_Click(object sender, EventArgs e)
{
    // 카메라가 꺼져있다면 카메라를 켜줌
    if (!isCameraRunning)
    {
        // 0번 카메라 시작
        video = new VideoCapture(0);
        video.FrameWidth = 640;
        video.FrameHeight = 480;
```

버튼 클릭을 통한 카메라 on/off 전환5/5

```
// 카메라가 정상적으로 열렸을 경우
                if (video.IsOpened())
                {
                    isCameraRunning = true;  // 카메라 상태 플래그 설정
                    timer1.Start(); //타이머 시작-> 프레임 주기적으로 읽음
                    btn_Toggle.Text = "Stop Camera";  // 버튼 텍스트 변경
                }
                else
                {
                    // 카메라 열기에 실패했을 때 사용자에게 메시지 표시
                    MessageBox.Show("카메라를 열 수 없습니다.");
                }            }
            else
            {
                // 카메라가 실행 중인 경우: 카메라 정지
                timer1.Stop();
                video.Release(); // 카메라 장치 해제
                pictureBox1.Image = null; // 화면 정리
                isCameraRunning = false;  // 카메라 상태 플래그 초기화
                btn_Toggle.Text = "Start Camera";  // 버튼 텍스트 초기화
            }
        }    }    }
```

동기식 코드의 특징

항목	설명
UI Timer 사용(제어방식)	System.Windows.Forms.Timer를 사용하여 일정 간격으로 카메라 프레임을 읽음
UI 스레드에서 실행	Timer_Tick 이벤트는 UI스레드에서 실행(프레임 읽기+이미지 갱신 모두 UI에서 수행)
동기 방식	내부적으로 await없이 전부 동기로 실행
병목 가능성	카메라 프레임 읽기나 Bitmap변환이 느릴 경우 UI멈춤이나 끊김 발생 가능
코드의 구현	작성이 간단하고 타이머 이벤트만 연결하면 됨 단순한 앱 또는 짧은 연산에 유리

```
// 타이머의 Tick 이벤트를 이용해 일정 간격으로 프레임 읽기
private void timer1_Tick(object sender, EventArgs e)
{
    video.Read(mImage);
    pictureBox1.Image = BitmapConverter.ToBitmap(mImage);
}
```

비동기식 코드의 특징

항목	설명
백그라운드 스레드에서 실행	Task.Run()을 통해 캡쳐 루프가 별도의 스레드에서 실행
UI는 Invoke로 접근	이미지 갱신만 UI 스레드에서 Invoke()로 수행하고, 나머지는 백그라운드
async/await 기반	비동기적으로 실행되어 UI 응답성이 향상됨
제어방식	CancellationToken을 이용해 종료 제어가 편리함(비동기 루프 + 취소 토큰)
코드의 구현	기존보다 코드 구조가 조금 복잡 실질적으로는 더 안정적이고 확장성이 좋음 실시간 처리, 고속 카메라 스트림 등

- Invoke란? 델리게이트가 참조하고 있는 메서드를 호출하는 것
  델리게이트?메서드에 대한 참조
  이벤트는 델리게이트를 기반으로 구현되기 때문에 이벤트가 발생할 때 내부적으로 Invoke가 호출되어 연결된 메서드(핸들러)를 실행
- Invoke를 사용하는 이유
  이벤트 실행 : 이벤트에 등록된 모든 메서드를 호출하기 위해 사용
  다중 핸들러 지원 : 하나 이상의 메서드를 한 이벤트에 등록할 수 있고 모두 순차적으로 호출됨
  예외 방지 : ?.Invoke 형태로 null이 아닌 경우에만 안전하게 호출 가능(null 체크)
  명시적 호출 : 델리게이트나 이벤트를 명시적으로 호출 할 수 있음

비동기식

- 스레드(Thread) : 프로그램에서 동시에 실행되는 흐름
  - 한쪽에서는 UI 처리, 다른 쪽에서는 카메라 프레임 읽기
  - WinForms 앱은 UI 스레드 하나로 실행

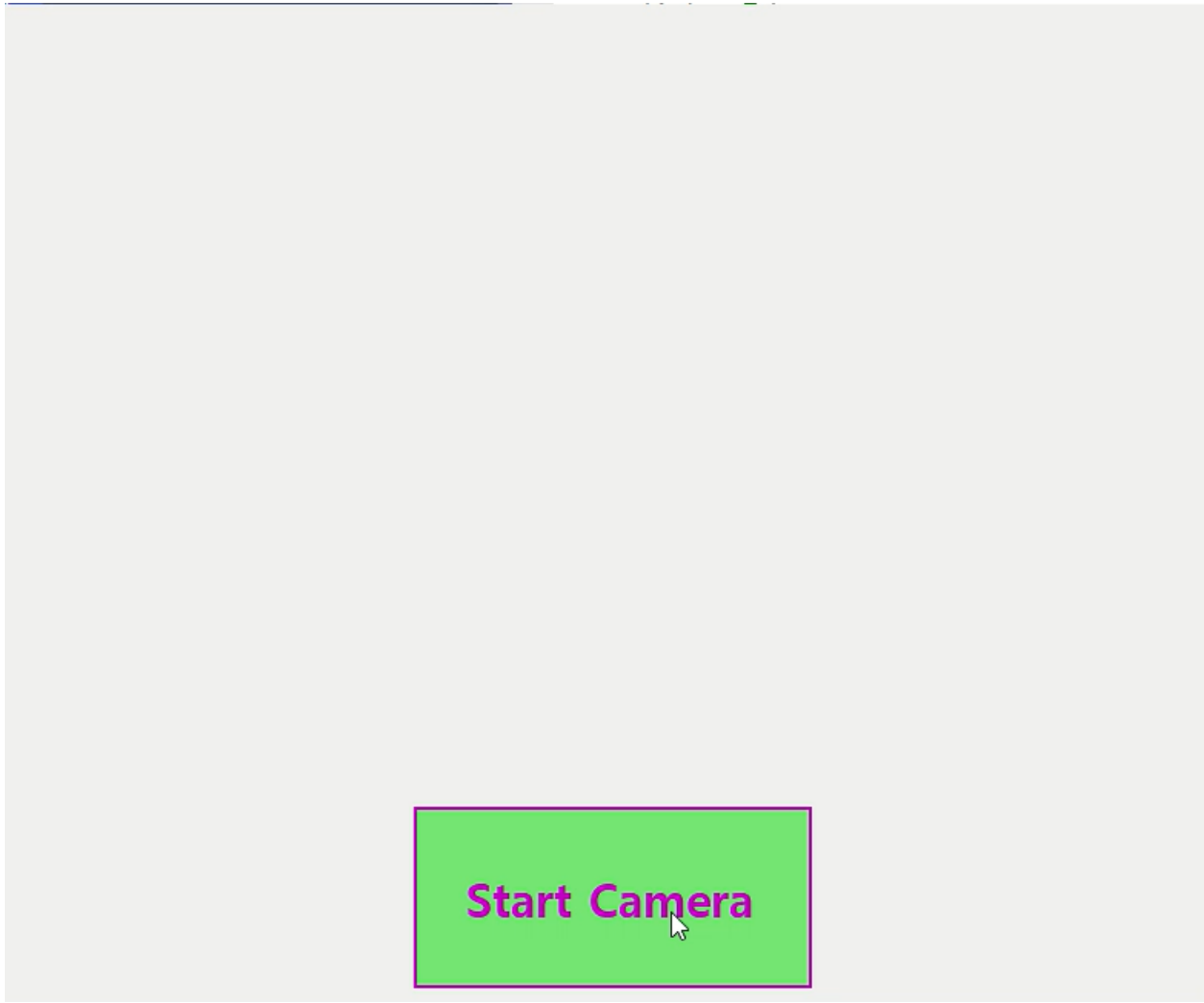
여러개 카메라

- 스레드 풀(Thread Pool)이란?
  - .NET에서 제공하는 재사용 가능한 작업용 스레드 모음
  - 작업이 필요할 때마다 새 스레드를 만들지 않고, 미리 만들어 둔 스레드를 빌려서 사용
  - 성능과 자원 낭비를 줄일 수 있음
  - .NET에서 비동기 작업이나 병렬 작업을 효율적으로 실행하기 위해 사용하는 스레드 관리 방식

구성요소	설명
Task.Run(...)	작업을 **스레드 풀 스레드에서 비동기적으로 실행** 무거운 작업을 UI 스레드가 아닌 **백그라운드**에서 실행해줌
await()	작업이 끝날 때까지 기다림 (비동기적)
cts.Token	취소 요청을 위한 토큰 전달
CaputreLoop(...)	실제로 카메라 프레임을 반복해서 읽는 작업

버튼 클릭을 통한 카메라 on/off 전환-비동기식

- 비동기식 구현
  - 비동기식 루프 구현
  - 취소 토큰 추가



버튼 클릭을 통한 카메라 on/off 전환-비동기식1/5

```
// OpenCV를 .NET 환경에서 사용할 수 있도록 해주는 확장 메서드 포함
using OpenCvSharp.Extensions;
using OpenCvSharp;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;
using System.Threading.Tasks;

namespace Nuget_opencvsharp
{
    public partial class Form1 : Form
    {
        // 카메라를 제어하는 VideoCapture 객체
        OpenCvSharp.VideoCapture video;

        // 비동기 작업 취소를 위한 토큰 선언
        private CancellationTokenSource cts;

        // 카메라 실행 여부를 판단하기 위한 상태 플래그
        private bool isCameraRunning = false; public Form1()
        {
            InitializeComponent();
        }
```

버튼 클릭을 통한 카메라 on/off 전환-비동기식2/5

```
 private void Form1_Load(object sender, EventArgs e)
 {
        btn_Toggle.Text = "Start Camera";
}
 private void Form1_FormClosing(object sender, FormClosingEventArgs e)
 {
     //timer1.Stop();   // 타이머 중지 (프레임 읽기 멈춤)
     cts?.Cancel();     // 캡처 루프 중단 요청
     video?.Release();  // 카메라 장치 해제
     //mImage.Dispose();   // Mat 메모리 해제
 }
```

버튼 클릭을 통한 카메라 on/off 전환-비동기식3/5

```
private async void btn_Toggle_Click(object sender, EventArgs e)
{
    if (!isCameraRunning)
    {
        // 카메라 시작

        video = new VideoCapture(0); // 기본 카메라 장치 열기
        video.FrameWidth = 640;      // 프레임 가로 해상도 설정
        video.FrameHeight = 480;     // 프레임 세로 해상도 설정

        if (video.IsOpened()) // 카메라 열기에 성공했는지 확인
        {
            isCameraRunning = true;
            btn_Toggle.Text = "Stop Camera";

            // 비동기 루프 제어를 위한 취소 토큰 생성
            cts = new CancellationTokenSource();
```

버튼 클릭을 통한 카메라 on/off 전환-비동기식4/5

```
 try
        {
            // 별도 스레드에서 캡처 루프 실행
            await Task.Run(() => CaptureLoop(cts.Token), cts.Token);
        }
        catch (OperationCanceledException)
        {
            // 사용자 또는 종료 이벤트에 의해 취소될 경우 예외 무시
        }     }
    else
    {
        // 카메라 오픈 실패 시 알림
        MessageBox.Show("카메라를 열 수 없습니다.");
    }   }
else
    {
        // 카메라 정지

        cts?.Cancel();                  // 캡처 루프 중단 요청
        isCameraRunning = false;        // 상태 변수 false로 설정
        btn_Toggle.Text = "Start Camera"; // 버튼 텍스트 되돌리기
        video?.Release();               // 카메라 장치 해제
        pictureBox1.Image = null;       // 화면에 남은 이미지 제거
    }
}
```

버튼 클릭을 통한 카메라 on/off 전환-비동기식5/5

```
private void CaptureLoop(CancellationToken token)
        {
            using var frame = new Mat(); // 캡처한 프레임을 저장할 OpenCV 이미지

            while (!token.IsCancellationRequested) // 취소 요청 전까지 반복
            {
                video.Read(frame); // 프레임 읽기

                if (!frame.Empty()) // 프레임이 유효하면 화면에 표시
                {
                    // WinForms UI는 UI 스레드에서만 접근 가능하므로 Invoke 사용, 람다식 정의
                    Invoke(new Action(() =>
                    {
                        pictureBox1.Image?.Dispose(); // 이전 이미지 해제
                        pictureBox1.Image = BitmapConverter.ToBitmap(frame); // 새 프레임 표시
                    } ) );
                }
                Thread.Sleep(33); // 약 30fps 유지를 위한 대기 시간 (~33ms)
            }
        }
    }
}
```